

Koa-action: Fast and Consistent Structured Decision Making with Generative LLMs*

Shenghong Dai^{1,2}, Shiva Kumar Pentyala¹, Yingchi Liu¹, Shubham Mehrotra¹,
Suman Banerjee², James Zhu¹, Bin Bi¹, Sitaram Asur¹, Phil Mui¹

¹Salesforce AI, ²University of Wisconsin–Madison

Abstract

Industry applications often demand low-latency classification, yet current large language model (LLM) approaches remain poorly suited for latency-critical applications. Existing prompting and constrained decoding produce verbose, multi-token outputs that require expensive token-by-token generation, while encoder-based models achieve faster inference but sacrifice the task flexibility. We propose Koa-action, a framework for low-latency *atomic actions*—fast, single-step decisions such as classification, semantic endpointing, Boolean checks, and scoring—formulated as constrained generation with single-token outputs. By introducing atomic label tokens and applying supervised fine-tuning, our method reduces classification to a deterministic one-step decoding problem. Across standard benchmarks, Koa-action delivers competitive accuracy with consistently low and stable latency. On a **production intent-routing benchmark**, Koa-action reaches **85.5%** accuracy—competitive with the strongest frontier models (Claude-4.8-Opus, Gemini-Pro-3.1) and ahead of GPT-5 and Gemini-2.5-Pro—while answering in about **half a second**—several-fold faster than every frontier model (up to $\sim 7.5\times$ at the median) under identical serving conditions. Against the dedicated single-token system Jev/TypeSafe, Koa-action is competitive on accuracy and faster at the median, while also handling multimodal inputs and multi-label outputs that single-label text systems do not.

1 Introduction

Industry applications rely heavily on classification for a wide range of decisions, from sentiment analysis (Pang et al., 2008) and intent recognition (Goo et al., 2018; Chen et al., 2019) to customer support and interactive dialogue agents. As these applications increasingly operate on multimodal data—

combining text and vision—there is growing demand for unified models that can handle diverse input modalities while maintaining efficiency in latency-sensitive environments (Wang et al., 2024).

Current approaches to classification with large language models (LLMs) face significant limitations, particularly in latency-critical applications. Prompt-based methods, while intuitive, often produce verbose, multi-token responses that require additional parsing, which introduces substantial inference overhead. More importantly, they provide no guarantee that outputs will be single tokens: a request such as “classify this review as positive or negative” can yield explanatory sentences or multi-token paraphrases rather than clean categorical labels. Even with constrained decoding techniques (Geng et al., 2023) that restrict outputs to valid label strings, models still rely on token-by-token generation. This scales poorly with label vocabulary size and leads to unpredictable latency variations.

This latency challenge is particularly acute in real-world deployment scenarios where classification must occur at scale with strict response time requirements. Traditional encoder-based approaches (e.g., BERT with classification heads) offer predictable, low-latency inference but lack the flexibility and generalization capabilities that make modern LLMs attractive for complex reasoning tasks, since they require task-specific architectures and dataset-specific fine-tuning.

In this work, we propose Koa-action, an approach that bridges this gap by treating classification as a constrained generation task with **single-token outputs**. More broadly, we view classification as one member of a family of *atomic actions*—fast, “System 1” decisions that software can consume directly, such as classification, semantic endpointing, Boolean checks, and scoring—all of which reduce to producing a single typed token in one step. Our key insight is to introduce atomic special tokens (e.g., [control_1]) for each class,

*An earlier version of this work was made available on [OpenReview](#) in September 2025

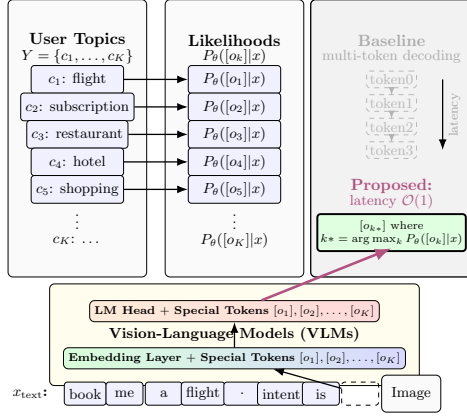


Figure 1: **Overview of our framework.** Inputs are processed by an LLM that directly outputs an atomic special token for the target class. Unlike baseline prompting with multi-token decoding, the framework performs classification in a single $\mathcal{O}(1)$ decode step (independent of output length) and supports zero-shot adaptation.

enabling the model to produce decisions in exactly one generation step. As illustrated in Figure 1, this design not only eliminates multi-token decoding overhead but also enables *zero-shot classification*: by reassigning label tokens at inference time, the model seamlessly adapts to new tasks without task-specific retraining.

We demonstrate this approach using supervised fine-tuning on large language models, creating a unified framework that handles both text-only and multimodal classification tasks. Our method rests on three central pillars: **accuracy**, **latency**, and **generality**. By constraining the output space to a finite set of learned tokens, we achieve deterministic single-step inference while maintaining the semantic understanding capabilities of large pre-trained models. Our contributions are threefold:

1. **A unified single-token classification framework** that integrates single-step constrained generation, randomized label assignment, and multimodal inputs into one deployable interface, removing multi-token decoding overhead while retaining the broad generalization of decoder LLMs.
2. **Frontier-level accuracy at a fraction of the latency.** Under an identical, directly comparable serving setup, Koa-action matches the accuracy of the strongest frontier LLMs (Claude-4.8-Opus, Gemini-Pro-3.1) while answering in sub-second time—where those frontier models take one to four seconds at

the median—with a near-flat tail that holds steady as the label space grows.

3. **Validation across real deployment settings, alongside comprehensive public benchmarks.** Beyond standard public datasets (text-only and multimodal), we evaluate Koa-action on a real production workload from a conversational agent—a de-identified intent-routing task whose label inventory evolves over time—and on a latency-critical synthetic voice turn-taking / endpointing task where sub-second tail latency is a hard requirement, showing the framework holds under real production constraints, not just academic ones.

2 Related Work

2.1 Prompt-Based Classification with LLMs

LLMs are often adapted for classification via prompting and in-context learning (further strengthened by instruction tuning) (Brown et al., 2020; Sun et al., 2023; Ouyang et al., 2022; Wei et al., 2021), yet their outputs can remain free-form and multi-token, complicating structured integration and increasing inference cost.

To mitigate this, methods such as PET and LM-BFF reformulate classification as a cloze task with *verbalizers* that map each class to a natural-language string and fine-tune the model on small labeled sets (Schick and Schütze, 2020; Gao et al., 2020). Beyond discrete verbalizers, *continuous prompting* methods have been proposed: Prompt Tuning learns soft prompt embeddings optimized for a target task (Lester et al., 2021; Ding et al., 2023), while Prefix Tuning prepends trainable key-value vectors to each transformer layer (Li and Liang, 2021). Although effective, these approaches are typically developed for few-shot learning scenarios with limited labeled data; in contrast, our work targets settings with more training examples.

Recent work uses *constrained decoding* to restrict generations to valid label strings (Geng et al., 2023), and a mature ecosystem of structured-generation engines now enforces such constraints efficiently at the inference layer (Willard and Louf, 2023; Dong et al., 2024; Zheng et al., 2023). However, these methods constrain *which* tokens may be emitted while still decoding label strings token by token, so their cost grows with label-string length and can be inefficient for large label spaces or multimodal settings; imposing output formats can also degrade task performance (Tam et al., 2024). We

instead use a finite set of atomic label tokens, so that a valid decision is guaranteed in a single decoding step regardless of how the label reads as text.

2.2 Encoder Fine-Tuning for Classification

A widely adopted paradigm emphasizes *fine-tuning encoder models* with a task-specific classifier head. Transformer encoders such as BERT and its successors (e.g., RoBERTa, DeBERTa) have become the standard for text classification (Devlin et al., 2019; Liu et al., 2019; He et al., 2020). For multi-modal classification, encoder-style fusion models extend this paradigm by incorporating vision or audio encoders and cross-modal attention modules, as in MulT and MAG-BERT (Tsai et al., 2019; Rahman et al., 2020). These approaches are efficient and effective, but they require task-specific classifier heads and lack the flexibility of LLMs. In contrast, Koa-action shows that *decoder-style VLMs*, adapted with single-token label spaces, can match or surpass encoder baselines on multimodal datasets while retaining the generalization of decoder LLMs, and offers a *systematic framework* validated in a real production deployment. A complementary line of work instead repurposes decoder LLMs as encoders for classification and embedding, either by extracting last-token representations for a lightweight head or by unlocking bidirectional attention (BehnamGhader et al., 2024; Yousefmandi and Cooney, 2025); these methods still attach a separate task-specific head, whereas Koa-action keeps a single generative interface and emits the decision as one output token.

2.3 Single-Token and Restricted-Vocabulary Classification

A recent line of work—concurrent with and independent of ours—reformulates classification so that a decision is read out in a single forward pass rather than generated token by token. Most closely related, SALSA (Berdichevsky et al., 2025) restricts the output distribution to per-class tokens and adapts the model with parameter-efficient fine-tuning, reporting strong accuracy at low cost. Our framework differs in two respects that matter for deployment: we *randomize the label-to-token assignment during training*, decoupling each control token from fixed class semantics and enabling zero-shot adaptation to new or relabeled class sets at inference; and we treat *latency as a first-class object of study*, reporting full P50/P95 tail-latency un-

der an identical, directly comparable serving setup against a broad panel of frontier LLMs, which prior single-pass classifiers omit.

This single-token, typed-decision paradigm has also attracted industry attention. Commercial systems such as TypeSafe AI’s “System One” models target instant, low-latency decisions that software can consume directly (TypeSafe AI, 2026), and independent reproductions report calibrated one-pass readouts from small LoRA-tuned backbones (Ren et al., 2026). These efforts underscore the practical demand for the problem we study, but they differ from Koa-action in scope and evidence: the commercial training recipe is proprietary, and—most relevant to our setting—they address *single-label, text-only* typed decisions and do not target multi-modal inputs or multi-label outputs. The approach we describe was publicly disclosed and deployed earlier (OpenReview, September 2025; Salesforce Engineering, November 2025); this paper formalizes it, extends it to multimodal and multi-label classification, and characterizes its latency behavior against strong baselines under real production constraints.

3 Methodology

3.1 Problem Formulation

We formalize classification as a supervised learning problem. Each training instance consists of an input tuple

$$x = (x^{\text{text}}, x^{\text{vis}}),$$

where x^{text} denotes the textual input and x^{vis} represents the vision modality, which may include static images or short video segments. Not all instances contain both modalities; missing modalities are treated as empty. We aim to learn a mapping

$$f_{\theta} : (x^{\text{text}}, x^{\text{vis}}) \mapsto y,$$

where $y \in \mathcal{Y}$ is a categorical label drawn from a predefined set of classes $\mathcal{Y} = \{c_1, c_2, \dots, c_K\}$.

Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, the learning goal is to estimate model parameters θ that minimize the expected classification loss:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathcal{L}(f_{\theta}(x), y),$$

where $\mathcal{L}$ is the cross-entropy loss over labels. This formulation generalizes unimodal classification to a multimodal setting, where textual and visual inputs are jointly leveraged for improved prediction.

3.2 Single-Token Classification with Atomic Label Tokens

Our proposed framework Koa-action is model-agnostic and can be applied to a wide range of LLMs and VLMs, including the Gemma, Mistral, and GPT-OSS architectures we use here, among others. The general principle is to treat classification as a constrained generation task, where the model is guided to produce a single special token corresponding to the target label.

Special Tokens for Labels. For each classification category $c_k \in \mathcal{Y}$, we introduce a unique special token $[o_k]$. Conventional choices such as digits or short text labels are problematic: they can appear naturally in the input, causing ambiguity, and larger indices (e.g., “100”) are split into multiple subword tokens, leading to multi-step decoding and higher latency. To provide grounding, a natural-language description of each class is included in the system prompt (see Appendix G for prompt templates). During fine-tuning, the model is trained to map $(x^{\text{text}}, x^{\text{vis}})$ directly to the correct special token.

Let $\mathcal{Y} = \{c_1, \dots, c_K\}$ denote the label set and let $\mathcal{V}$ be the tokenizer vocabulary. We augment the vocabulary with K atomic output symbols:

$$\Omega = \{[o_1], [o_2], \dots, [o_K]\}, \quad \tilde{\mathcal{V}} = \mathcal{V} \cup \Omega.$$

Each $[o_k]$ is a dedicated *control token* corresponding to class c_k , with a trainable embedding $e_k \in \mathbb{R}^d$. These rows are appended to the model’s embedding matrix and are updated during fine-tuning, together with the adapted model weights. In implementation, we pre-allocate a pool of up to 500 reserved control tokens, allowing the framework to support classification tasks with as many as 500 labels without retraining.

Why single-token outputs. If labels are represented as natural-language strings s_k (e.g., “book a flight”), a subword tokenizer typically produces a variable-length sequence

$$\tau(s_k) = (v_1, \dots, v_{m_k}), \quad m_k \geq 1,$$

where $\tau(\cdot)$ denotes the tokenization function. This variability requires loss over multiple decoding steps and depends on segmentation. In contrast, our design assigns each class a single atomic symbol $\omega_k = [o_k] \in \Omega$, which is guaranteed to decode as exactly one token. This yields three benefits: (i) the output space collapses to K symbols, (ii) ambiguity from label strings that may appear in the input

is eliminated, and (iii) decoding and evaluation are simplified to a deterministic one-step classification.

Randomized label assignments. To prevent memorization of static token–label associations, we do not fix a permanent mapping between special tokens and semantic classes. Instead, during preprocessing we randomly shuffle the correspondence between classes and control tokens across training instances. This design encourages the model to rely on the contextual descriptions of labels provided in the prompt, rather than memorizing token identities. As a result, the model learns to infer the correct output token from the input context, which improves robustness and generalization across datasets and label spaces. Without randomization, models tended to overfit to token IDs and failed to generalize to new mappings.

Separation from context. We reserve a sentinel namespace for Ω , ensuring that these tokens are never decomposed into subwords and never occur in the input text. Formally, $\Omega \cap \mathcal{V} = \emptyset$ and $\Omega \cap \tau(x) = \emptyset$ for any input x . In practice, this is enforced by registering $[o_k]$ as special tokens in the tokenizer, so that they are available to the model during output generation.

Training objective (single position). Given input $x = (x^{\text{text}}, x^{\text{vis}})$ with gold label $k \in \{1, \dots, K\}$, the model is required to emit exactly one control token $[o_k]$. Let h_T denote the decoder state at the output position (the single assistant step). We compute logits restricted to Ω :

$$z_j = (Wh_T + b)_j, \\ P_\theta([o_j] | x) = \frac{\exp(z_j)}{\sum_{i=1}^K \exp(z_i)}, \quad j \in \{1, \dots, K\}.$$

The loss is standard cross-entropy over this single prediction:

$$\mathcal{L}(x, k) = -\log P_\theta([o_k] | x).$$

All preceding tokens (system/user prompts and any in-context descriptions) are masked out of the loss, so classification supervision is concentrated solely on the final output position. In implementation, we construct a binary loss mask: the position of the response token is assigned its class label, while all other positions are set to -100 , which the cross-entropy loss ignores.

Inference rule. At test time, decoding reduces to a single restricted argmax:

$$k^* = \arg \max_{k \in \{1, \dots, K\}} P_\theta([o_k] \mid x).$$

Equivalently, we set `max_new_tokens = 1` and restrict the decision space to Ω , ensuring the model outputs exactly one control token (e.g., $[o_{k^*}]$) rather than a multi-token string. This deterministic procedure avoids token-by-token generation: the *decode* stage is a single forward pass whose cost is independent of output length, unlike autoregressive decoding. The *prefill* stage still scales with input and prompt length, so our $\mathcal{O}(1)$ claim refers specifically to decoding.

3.3 Extension to Multi-Label Classification

In the settings considered so far, each input is assigned exactly one class ($y \in \mathcal{Y}$). Many real-world applications are instead *multi-label*: a single input may belong to several classes at once, so the prediction target is a *subset* $Y \subseteq \mathcal{Y}$ rather than a single label. For instance, a customer utterance may simultaneously request an action, reference a prior case, and fall under a compliance category. Autoregressive approaches handle this awkwardly, because generating labels one at a time is known to suppress all but one label at each step, a bias that supervised fine-tuning tends to amplify (Ma et al., 2025). Our single-token formulation sidesteps this pathology by construction: the restricted softmax over Ω defined above already yields, in a single forward pass, a full probability distribution over the entire label-token set, so the relative evidence for *all* candidate labels is available simultaneously, without any sequential generation.

The key enabler is a change in *training supervision* rather than in architecture or decoding. Instead of supervising the restricted distribution toward a one-hot target concentrated on a single control token, we train the model to match a *soft target distribution* that spreads probability mass across the control tokens of all labels relevant to an instance. Concretely, the model minimizes a distribution-matching objective—a divergence between the predicted restricted distribution over Ω and this target—instead of the single-label cross-entropy used above. This teaches the model to allocate calibrated probability mass over co-occurring labels while preserving the single-pass, single-step decoding cost: multi-label prediction requires no additional generation steps and no architectural changes, only a

different training target. At inference, the same one-pass distribution is used to select the set of labels the model assigns sufficient probability to, rather than a single arg max. We omit the precise construction of the target distribution and the selection rule; our aim here is to show that the single-token framework *extends naturally* to multi-label settings, inheriting its latency and determinism guarantees.

4 Experiments

4.1 Datasets

We adapt each base model into a single-token classifier with supervised fine-tuning on a corpus of classification instances covering both text-only and multimodal inputs, and then evaluate the adapted models on the held-out benchmarks below; all reported evaluation datasets are disjoint from any data used for adaptation.

Evaluation benchmarks. We evaluate our approach on a diverse suite of multimodal and text-only classification benchmarks. For multimodal evaluation, we use the official test split of **MIntRec2.0** (Zhang et al., 2024), a large-scale benchmark for intent recognition in multimodal dialogues that combine text and vision. This dataset includes 30 fine-grained intent classes and requires reasoning over conversational context and multiple modalities, making it a challenging test of real-world multimodal classification. For text-only tasks, we evaluate on two widely used public sentiment benchmarks: **SST-2** (Socher et al., 2013), a binary sentiment analysis benchmark of movie reviews annotated as positive or negative, and **Amazon Reviews Polarity** (McAuley and Leskovec, 2013; Zhang et al., 2015), which contains millions of product reviews labeled as positive or negative and probes sentiment classification in a large-scale, noisy e-commerce domain. We evaluate on the *full* SST-2 test set; for Amazon Reviews Polarity, whose test set is much larger, we evaluate on 5k examples of the official test split. We further include an **intent routing** dataset derived from a production conversational agent, where each example is labeled with an intent that triggers a pre-defined routing action (mapping the request to a follow-up or tool call); we report only aggregate results.¹ Finally, to go beyond sentiment into a modern deployment setting, we add a **voice turn-taking / endpointing** benchmark (353 examples,

¹We cannot release this dataset due to privacy and contractual restrictions; all reported numbers are aggregated.

binary): given a partial conversation transcript, the model must decide whether the speaker has finished their turn (so the agent should respond) or is still speaking (so the agent should keep listening). We further report results on additional public benchmarks (DBpedia (Lehmann et al., 2015), Tweet-Topic, Banking77, and MTOP) in Appendix B. Example prompt templates are provided in Appendix G (Figures 9 to 12).

4.2 Baselines

We compare against both open-source models and commercial API systems. **Pretrained LLMs.** We adapt open-weight base models from the Gemma, Mistral, and GPT-OSS families, and also evaluate their un-adapted checkpoints to quantify the performance of large pretrained models without task-specific adaptation (exact checkpoints are given with each results table). **Commercial API models.** As strong performance benchmarks, we evaluate a broad panel of frontier LLMs spanning the major providers: GPT-4o, GPT-4.1, GPT-4.1-mini, and GPT-5 (OpenAI); Claude-4-Sonnet, Claude-5-Sonnet, and Claude-4.8-Opus (Anthropic); and Gemini-2.5-Pro and Gemini-Pro-3.1 (Google). **Dedicated single-token systems.** We further compare against Jev/TypeSafe, a commercial system purpose-built for single-token typed decisions, on the tasks where its interface applies. **Encoder-based models.** To contextualize against specialized architectures, we also report results for strong encoder-style multimodal baselines (e.g., MAG-BERT and MuT), following the MINTREC 2.0 benchmark protocol. For text benchmarks, we include: (i) a traditional linear classification head applied to BERT-base and RoBERTa-base (16-shot fine-tuning), and (ii) LM-BFF (Gao et al., 2020) with a RoBERTa-base backbone as a representative prompt-based few-shot classifier.

4.3 Evaluation metrics

We evaluate models along two complementary dimensions. **Classification accuracy.** The primary metric is accuracy, computed as the percentage of instances where the predicted control token matches the gold label. Accuracy is reported separately for each dataset. **Latency.** To capture serving efficiency, we report the median (P50) and tail (P95) latency of a single request. Comparing systems as different as our single-token classifier and commercial frontier LLMs requires care, so we access every model through the *same* serving inter-

face and issue requests one at a time in round-robin order: each model faces identical load and its measured end-to-end time is directly comparable to the others, with no batching and no client-side caching (§4.4.3). We separately examine how latency scales with batch size using our own vLLM-based inference framework (Kwon et al., 2023); Appendix D.2 presents a batch-scaling study demonstrating that the latency benefits of Koa-action persist even under large-batch inference settings.

4.4 Results

4.4.1 Multimodal evaluation results

We evaluate our fine-tuned models trained on the challenging MIntRec 2.0 dataset, which requires understanding multimodal dialogue contexts (text, image, video) for intent recognition.

MIntRec 2.0 (Multimodal Topic Classification).

Table 1 presents results on MIntRec 2.0. Evaluated zero-shot under a natural label-name prompt, base VLMs reach only $\sim 35\text{--}43\%$ accuracy, underscoring the difficulty of multimodal intent recognition without adaptation. After fine-tuning, performance improves markedly: Gemma-3-4B reaches **55.2%**, while Gemma-3-27B achieves **62.7%**. Crucially, this margin holds not only over the original GPT-4o (42.1%) and GPT-5 (48.5%) baselines but against the strongest frontier VLMs evaluated under the identical prompt *with* the same video frames: Gemini-Pro-3.1 (55.9%), Claude-4.8-Opus (52.9%), and Claude-5-Sonnet (49.8%) all remain below our fine-tuned Gemma-3-27B, and even our 4B Koa-action model (55.2%) surpasses every frontier VLM except Gemini-Pro-3.1. Even with the video frames available, frontier VLMs plateau well below our adapted models, consistent with our motivation that the visual signal needs task-specific multimodal adaptation to become usable.

Beyond this standard GPT-4o configuration, we also explore whether carefully designed class-index prompts can emulate our single-token interface; Appendix F.1 (*Alternative Prompting Baselines*) shows that such prompting-only variants still frequently produce multi-token outputs and achieve only 45.12% accuracy. To further strengthen this comparison, we additionally evaluate a constrained-decoding variant of GPT-4o, where all 30 intents are mapped to digit-only labels and decoding is restricted to this label set. This improves GPT-4o’s accuracy to 45.56%, but it still falls well short of Koa-action-adapted open-source models.

Model	Acc.
<i>Base VLMs (no adaptation, zero-shot)</i>	
Mistral-3-24B (Base)	35.17
Gemma-3-4B (Base)	36.65
Gemma-3-27B (Base)	43.29
<i>Frontier VLMs (zero-shot, +video)</i>	
GPT-4o	42.06
GPT-4.1	45.40
GPT-4.1-mini	45.99
GPT-5	48.45
Claude-4-Sonnet	48.55
Gemini-2.5-Pro	49.14
Claude-5-Sonnet	49.83
Claude-4.8-Opus	52.88
Gemini-Pro-3.1	55.93
<i>Encoder baselines (task-specific FT)</i>	
MAG-BERT	60.58
MuT	60.66
<i>Fine-tuned (Koa-action, ours)</i>	
Mistral-3-24B (FT, Koa-action)	49.34
Gemma-3-4B (FT, Koa-action)	55.19
Gemma-3-27B (FT, Koa-action)	62.72

Table 1: MIntRec 2.0 accuracy (%), all models given text + video. Base VLMs and frontier VLMs are evaluated zero-shot under the *same* natural-language prompt (list the 30 intent names, reply with one). The strongest zero-shot model (Gemini-Pro-3.1, 55.9%) still trails our fine-tuned Gemma-3-27B (62.7%); even our 4B Koa-action model (55.2%) surpasses every frontier VLM except Gemini-Pro-3.1.

Comparison with Encoder-Based Models. MAG-BERT and MuT, strong encoder-based multimodal baselines, achieve around 60.6% accuracy. Our fine-tuned Gemma-3-27B surpasses both, reaching **62.7%**, while also offering the benefits of a unified generalization modeling framework. However, these encoder-based models are fully fine-tuned specifically on MIntRec 2.0 with task-specific classifier heads tied to a fixed label space, and therefore do not generalize to new tasks or label sets without retraining. In contrast, Koa-action models preserve the generalization interface of decoder LLMs and support zero-shot adaptation across tasks; we further analyze this generality in Section 4.4.2. The competitive or superior performance of fine-tuned LLMs indicates that large decoder-based architectures, when fine-tuned on in-domain data, can match or outperform specialized encoder-based models while simultaneously enabling broader generalization and reasoning capabilities.

4.4.2 Text classification across domains

We evaluate on two widely used public sentiment benchmarks (SST-2, Amazon Reviews), an intent-routing dataset, and a voice turn-taking / endpointing benchmark. Evaluations on additional public benchmarks (DBpedia, TweetTopic, Banking77, and MTOP (Li et al., 2021)) are provided in the appendix (Appendix B and the scaling/cardinality analyses). We assess cross-domain transfer by evaluating our *pre-adapted* single-token classifiers zero-shot on these datasets, and compare them against a broad panel of frontier LLMs (also zero-shot), encoder/few-shot baselines, and an external commercial evaluator. Table 2 reports the per-dataset accuracy results, and Figure 2 summarizes end-to-end serving latency across the same models.

Notably, the intent-routing setting differs from public benchmarks in that the topic inventory evolves over time (e.g., new intents are introduced and old ones are merged or deprecated). This makes classifier-head baselines and dataset-specific verbalizers tied to a fixed label space and thus costly to maintain, whereas Koa-action’s reserved atomic label tokens support one-step routing with prompt-side label updates. On this intent-routing benchmark, Koa-action reaches **85.5%** accuracy, competitive with the frontier panel (within ~ 2 points of the strongest, Claude-4.8-Opus and Gemini-Pro-3.1), while serving each request in 0.53 s at the median (p95 0.63 s)— $\sim 1.9\text{--}7.5\times$ faster at the median (and $\sim 2.4\text{--}9.0\times$ at the P95 tail) than these frontier models under identical serving conditions (Figure 2). We next evaluate on public benchmarks with fixed label spaces.

As a basic encoder-style baseline, we include BERT-base and RoBERTa-base models equipped with a linear classification head, fine-tuned using 16 examples per class. As shown in Table 2, these linear-head models exhibit the weakest accuracy, and—being tied to a fixed label space—do not transfer to the intent-routing or voice-endpointing settings. To provide a stronger encoder-based comparison, LM-BFF augments RoBERTa with prompt-based verbalizers and in-context examples. LM-BFF clearly improves over linear-head fine-tuning, reaching 83.8–90.1% accuracy on SST-2 and Amazon Reviews. However, it still falls short of our Koa-action models, which reach 95.8% on SST-2 and 96.6% on Amazon Reviews. Moreover, LM-BFF relies on multi-token prompted decoding and, like the linear-head baselines, requires task-

Model	SST-2	Amazon	Intent Routing	Voice
<i>Encoder / few-shot baselines (task-specific)</i>				
BERT (linear head)	70.87	77.04	—	—
RoBERTa (linear head)	61.35	79.16	—	—
LM-BFF	90.14	83.76	—	—
<i>Ours — Koa-action single-token classifiers (zero-shot on every set)</i>				
Gemma-4-26B (FT, Koa-action)	95.76	96.60	81.43	94.33
GPT-OSS-20B (FT, Koa-action)	92.89	95.34	85.52	96.90
<i>Frontier LLMs (zero-shot)</i>				
GPT-4o	95.64	96.32	84.84	95.75
GPT-4.1	95.99	96.56	86.03	96.03
GPT-4.1-mini	94.61	96.28	69.34	94.62
GPT-5	95.64	96.40	83.99	96.88
Claude-4-Sonnet	95.41	96.78	86.03	97.17
Claude-5-Sonnet	95.76	97.28	85.69	96.88
Claude-4.8-Opus	96.33	97.14	87.73	96.60
Gemini-2.5-Pro	95.64	96.68	84.50	96.60
Gemini-Pro-3.1	96.79	97.42	87.05	95.18
<i>External evaluator (public + synthetic only)</i>				
Jev (TypeSafe)	96.10	96.80	—	93.20

Table 2: Text classification accuracy (%) on SST-2 (872 examples), Amazon Reviews (5k), a production intent-routing benchmark (587 utterances, with a label inventory that evolves over time), and a voice turn-taking / endpointing benchmark (353 examples, binary). **Every LLM row—ours and the frontier models—is evaluated zero-shot; no model in this table was trained on any of these datasets.** Our Koa-action single-token classifiers sit inside the frontier band on every set (within ~ 1 –2 points of the best model) and lead the smaller/deployable tier, at a fraction of the serving latency (Figure 2). Encoder / few-shot baselines are task-specific and are not applicable to the intent-routing benchmark (its label space evolves over time, requiring retraining) or to voice endpointing. Jev is not run on the intent-routing set (internal data). **Bold** = best per column.

specific fine-tuning for each dataset, so it does not transfer to new or evolving label sets.

Without task-specific adaptation, Koa-action demonstrates **robust cross-domain performance**. As shown in Table 2, our fine-tuned models are **competitive with the frontier panel** on sentiment classification (SST-2, Amazon Reviews) and stay within ~ 1 –2 points of the strongest frontier models on every set, while leading the smaller/deployable tier on the intent-routing benchmark. They match this accuracy at a small fraction of the serving latency—the subject of the next section (§4.4.3). Additional analyses on label-set cardinality and model scaling are deferred to Appendix D.1 and Appendix D.4.

We further examine whether the model operates under unseen label–token mappings (Appendix D.3); a consolidated accuracy–latency comparison is given in Figure 4.

4.4.3 Serving latency

Accuracy places Koa-action inside the frontier band; latency is where the single-token formulation separates from it. To make the comparison fair, we

access every model—Koa-action and all frontier LLMs—through the *same* serving interface and issue requests one at a time in round-robin order, so each model faces identical load and its measured time is directly comparable to the others; we report the honest end-to-end time of a single request (no batching, no client-side caching). Because Koa-action emits exactly one token, its end-to-end time is essentially its time-to-first-token; frontier models instead pay for multi-token generation, and that difference is the entire point.

Sub-second serving with a flat tail. On the intent-routing set (Figure 2), Koa-action answers in **0.53 s** at the median and **0.63 s** at the P95—from $\sim 1.9\times$ faster than the quickest frontier model (GPT-4.1-mini) to $\sim 7.5\times$ faster than the slowest (Gemini-Pro-3.1). Just as important as the median is the *shape* of the distribution: Koa-action’s tail is nearly flat (P95–P50 ≈ 0.10 s), whereas frontier models fan out by 0.5–1.7 s from median to P95, so the speed gap only widens at the tail. Under real load and retries it is the tail, not the median, that

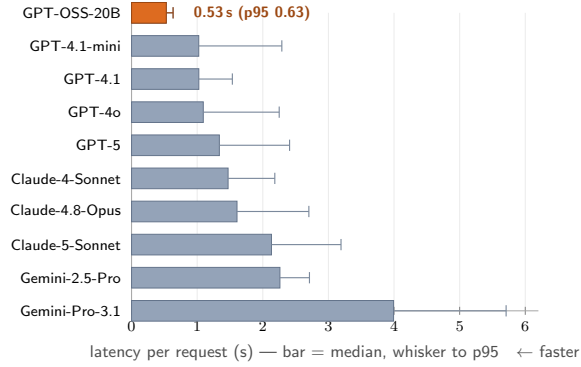


Figure 2: End-to-end per-request serving latency on the intent-routing set (587 utterances), median (p50) with a whisker to the p95 tail. Orange = our Koa-action single-token classifier; slate = frontier LLMs. Every model is accessed through the same serving interface and issued the same requests one at a time in round-robin order, so all models see identical load and their times are directly comparable. Our classifier answers in **0.53 s** (p50) — $\sim 1.9\times$ faster than the quickest frontier model (GPT-4.1-mini) and $\sim 7.5\times$ faster than the slowest (Gemini-Pro-3.1) — and has a near-flat tail (p95 = 0.63 s) where frontier models fan out by 0.5–1.7 s.

governs user-visible latency, so this gap widens in deployment.

Latency is independent of the task. Because a decision is always one forward step, Koa-action’s latency barely moves across tasks of very different label-space size and content: its median is 0.529 s on the evolving intent router, 0.526 s on binary SST-2 sentiment, and 0.527 s on voice turn-taking / endpointing (Figure 3; SST-2 in Appendix A, Figure 5)—the three medians fall within 3 ms of one another despite spanning a large evolving label set, binary sentiment, and a real-time transcript decision. Autoregressive baselines have no such guarantee—their cost scales with how many tokens they choose to emit, which is why their tails are both higher and more variable. This predictability, not just the average speed, is what makes the approach suitable for latency-critical routing.

The speedup is intrinsic to single-token decoding. A useful control is the external commercial single-token system Jev/TypeSafe, which on the voice endpointing task serves at 0.55 s median (P95 0.69 s), essentially tied with Koa-action (Figure 3)—and well ahead of every frontier LLM, which must decode over the running dialogue transcript and lands at 0.98–2.88 s at the median. This is the expected result and reinforces our thesis: the several-fold latency advantage comes from collapsing clas-

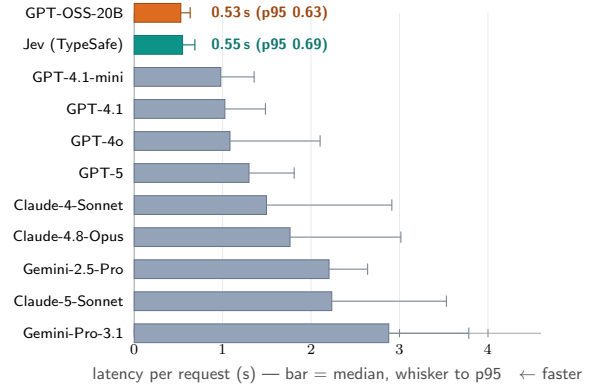


Figure 3: End-to-end per-request serving latency on the voice turn-taking / endpointing benchmark (353 examples), under the same round-robin, concurrency-1, shared serving-interface protocol as Figure 2. Orange = our Koa-action single-token classifier; teal = the external single-token system Jev/TypeSafe; slate = frontier LLMs. Koa-action answers in 0.53 s (p50 0.63 s), matching its latency on the intent-routing and SST-2 tasks almost to the millisecond, and the external single-token system Jev/TypeSafe is essentially tied (0.55 s, p95 0.69 s), while every frontier model must decode over a running dialogue transcript and lands at 0.98–2.88 s at the median.

sification into a single typed decision, not from any implementation trick particular to our system. Unlike Jev, however, Koa-action delivers this while also handling multimodal inputs and multi-label outputs.

Accuracy and latency together. Figure 4 unifies the two axes on the intent-routing benchmark, with every model measured under the identical round-robin protocol (single serving interface, one request at a time, end-to-end wall-clock). Our single-token classifier sits alone in the fast corner at 0.53 s and 85.5% accuracy, and anchors the fast end of the Pareto frontier, which it shares with just two other models: GPT-4.1, which edges ahead on accuracy (86.0%) at $\sim 1.9\times$ our latency, and Claude-4.8-Opus, the most accurate (87.7%) at $\sim 3\times$. A few more models are also more accurate but lie off the frontier because a frontier model already dominates them—Claude-4-Sonnet (86.0%), Claude-5-Sonnet (85.7%), and even Gemini-Pro-3.1 (87.1%), which pays $\sim 7.5\times$ our latency yet is still edged out on accuracy by Claude-4.8-Opus. The remaining frontier LLMs are strictly dominated by Koa-action itself—slower without being more accurate (GPT-4o, GPT-5, and Gemini-2.5-Pro all trail on both axes). No model is both faster and more accurate

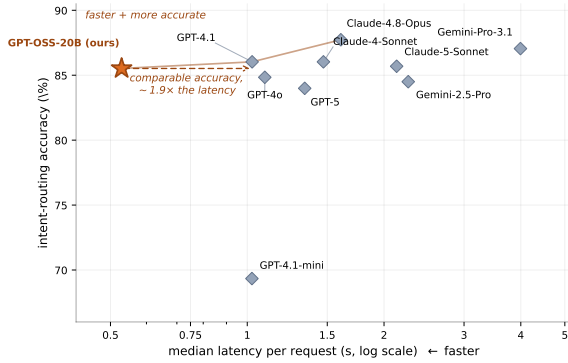


Figure 4: Accuracy vs. median serving latency on the intent-routing benchmark (log-scaled latency; lower-left of the accuracy axis omitted). All points use the same round-robin serving protocol. Our single-token classifier (orange star) occupies the fast, high-accuracy corner; the orange curve traces the Pareto frontier (Koa-action, GPT-4.1, Claude-4.8-Opus), and the dashed arrow marks the nearest frontier point of comparable accuracy (GPT-4.1), which runs at $\sim 1.9\times$ our latency. No model is both faster and more accurate than Koa-action.

than Koa-action, which places our system at the knee of the frontier rather than at either extreme.

5 Conclusion

We introduced Koa-action, a framework for structured decision making that casts classification as single-step constrained generation using atomic label tokens and supervised fine-tuning. Under an identical, directly comparable serving setup, Koa-action matches the accuracy of the strongest frontier LLMs across sentiment, intent routing, and multimodal intent recognition, yet answers in sub-second time with a near-flat tail—and, because every decision is a single decode step, its latency stays essentially constant (0.53 s at the median) across tasks whose label spaces and inputs differ sharply. This combination of frontier-level accuracy, several-fold lower latency, and task-independent predictability is what makes the framework practical under real production constraints; extending beyond text and vision and improving multilingual generalization are key directions.

Ethical Considerations

The intent-routing evaluation uses de-identified interactions from a production conversational agent. We report only aggregate metrics, include no raw utterances, and cannot release the data due to privacy and contractual restrictions. The intended use

is intent routing, not user profiling or high-stakes automated decisions.

References

- Dimosthenis Antypas, Asahi Ushio, Jose Camacho-Collados, Vitor Silva, Leonardo Neves, and Francesco Barbieri. 2022. Twitter topic classification. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 3386–3400.
- Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. [LLM2Vec: Large language models are secretly powerful text encoders](#). *arXiv preprint arXiv:2404.05961*.
- Ruslan Berdichevsky, Shai Nahum-Gefen, and Elad Ben Zaken. 2025. [SALSA: Single-pass autoregressive LLM structured classification](#). *arXiv preprint arXiv:2510.22691*.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. 2020. [Efficient intent detection with dual sentence encoders](#). In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, pages 38–45, Online. Association for Computational Linguistics.
- Qian Chen, Zhu Zhuo, and Wen Wang. 2019. Bert for joint intent classification and slot filling. *arXiv preprint arXiv:1902.10909*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186.
- Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, and 1 others. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature machine intelligence*, 5(3):220–235.
- Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Pan, and Tianqi Chen. 2024. [XGrammar: Flexible and efficient structured generation engine for large language models](#). *arXiv preprint arXiv:2411.15100*.
- Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik Krzemiński,

- Genta Indra Winata, Saba Sturua, Saiteja Utpala, Mathieu Ciancone, Marion Schaeffer, Gabriel Sequeira, Diganta Misra, Shreeya Dhakal, Jonathan Rystrom, Roman Solomatin, and 67 others. 2025. [Mmteb: Massive multilingual text embedding benchmark](#). *arXiv preprint arXiv:2502.13595*.
- Tianyu Gao, Adam Fisch, and Danqi Chen. 2020. Making pre-trained language models better few-shot learners. *arXiv preprint arXiv:2012.15723*.
- Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured nlp tasks without finetuning. *arXiv preprint arXiv:2305.13971*.
- Chih-Wen Goo, Guang Gao, Yun-Kai Hsu, Chih-Li Huo, Tsung-Chieh Chen, Keng-Wei Hsu, and Yun-Nung Chen. 2018. Slot-gated modeling for joint slot filling and intent prediction. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 753–757.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2020. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.
- Jens Lehmann, Robert Isele, Max Jakob, Anja Jentzsch, Dimitris Kontokostas, Pablo N Mendes, Sebastian Hellmann, Mohamed Morsey, Patrick Van Kleef, Sören Auer, and 1 others. 2015. Dbpedia—a large-scale, multilingual knowledge base extracted from wikipedia. *Semantic web*, 6(2):167–195.
- Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059.
- Haoran Li, Abhinav Arora, Shuohui Chen, Anchit Gupta, Sonal Gupta, and Yashar Mehdad. 2021. MTOP: A comprehensive multilingual task-oriented semantic parsing benchmark. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 2950–2962.
- Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Marcus Ma, Georgios Chochlakis, Niyantha Maruthu Pandiyan, Jesse Thomason, and Shrikanth Narayanan. 2025. [Large language models do multi-label classification differently](#). *arXiv preprint arXiv:2505.17510*.
- Julian McAuley and Jure Leskovec. 2013. Hidden factors and hidden topics: understanding rating dimensions with review text. In *Proceedings of the 7th ACM conference on Recommender systems*, pages 165–172.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2022. [Mteb: Massive text embedding benchmark](#). *arXiv preprint arXiv:2210.07316*.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.
- Bo Pang, Lillian Lee, and 1 others. 2008. Opinion mining and sentiment analysis. *Foundations and Trends® in information retrieval*, 2(1–2):1–135.
- Wasifur Rahman, Md Kamrul Hasan, Sangwu Lee, Amir Zadeh, Chengfeng Mao, Louis-Philippe Morency, and Ehsan Hoque. 2020. Integrating multimodal information in large pretrained transformers. In *Proceedings of the conference. Association for computational linguistics. Meeting*, volume 2020, page 2359.
- Simiao Ren, Kidus Zewde, Xingyu Shen, Yuchen Zhou, Dennis Ng, Ankit Raj, Tommy Duong, Yuxin Zhang, and Neo Tiangratanakul. 2026. [Open-jev judgments on CallScreenBench: Calibrated one-pass scam screening with a small language model](#). *arXiv preprint arXiv:2609.23959*.
- Timo Schick and Hinrich Schütze. 2020. Exploiting cloze questions for few shot text classification and natural language inference. *arXiv preprint arXiv:2001.07676*.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.
- Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. 2023. Text classification via large language models. *arXiv preprint arXiv:2305.08377*.
- Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. [Let me speak freely? a study on the impact of format restrictions on performance of large language models](#). *arXiv preprint arXiv:2408.02442*.

Yao-Hung Hubert Tsai, Shaojie Bai, Paul Pu Liang, J Zico Kolter, Louis-Philippe Morency, and Ruslan Salakhutdinov. 2019. Multimodal transformer for unaligned multimodal language sequences. In *Proceedings of the conference. Association for computational linguistics. Meeting*, volume 2019, page 6558.

TypeSafe AI. 2026. [Introducing system one models and Jev](#). TypeSafe AI Blog.

Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, and 1 others. 2024. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*.

Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. 2021. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*.

Brandon T. Willard and Rémi Louf. 2023. [Efficient guided generation for large language models](#). *arXiv preprint arXiv:2307.09702*.

Amirhossein Yousefiramandi and Ciaran Cooney. 2025. [Fine-tuning causal LLMs for text classification: Embedding-based vs. instruction-based approaches](#). *arXiv preprint arXiv:2512.12677*.

Hanlei Zhang, Xin Wang, Hua Xu, Qianrui Zhou, Kai Gao, Jianhua Su, Wenrui Li, Yanting Chen, and 1 others. 2024. Mintrec2. 0: A large-scale benchmark dataset for multimodal intent recognition and out-of-scope detection in conversations. *arXiv preprint arXiv:2403.10943*.

Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. [SGLang: Efficient execution of structured language model programs](#). *arXiv preprint arXiv:2312.07104*.

SUMMARY OF THE APPENDIX

This appendix contains additional details for the “*Turning Generative LLMs into Low-Latency and Consistent Classifiers*”. This is organized as follows:

- [§A Serving Latency on SST-2](#)
- [§B Additional Benchmarks](#)
- [§C Ablation Study](#)
- [§D Robustness and Generalization Analysis](#)
 - [D.1 Effect of Label-Set Sizes](#)
 - [D.2 Effect of Batch Size](#)
 - [D.3 Control-Token Permutations](#)
 - [D.4 Scaling Analysis](#)
- [§E Data Construction](#)
 - [E.1 Dataset Details](#)
 - [E.2 Implement Details](#)
- [§F Preliminary Study](#)
 - [F.1 Alternative Prompting Baselines](#)
- [§G Prompt Configuration](#)

A Serving Latency on SST-2

Figure 5 reports the same round-robin, single-concurrency, shared serving-interface latency protocol used for the intent-routing set in the main paper (Figure 2), here on the public SST-2 benchmark (872 examples); the companion voice end-pointing measurement is shown in the main text (Figure 3). The picture is identical to the intent-routing and voice settings: Koa-action serves at 0.53 s median / 0.63 s P95 on SST-2, essentially unchanged from its intent-routing latency despite the very different task, and the external single-token system Jev/TypeSafe is essentially tied (0.55 s / 0.66 s). Every frontier LLM is markedly slower and carries a wider P95 tail: they run from GPT-4.1-mini at 0.92 s median up to Gemini-Pro-3.1 at 3.32 s. This confirms that Koa-action’s latency advantage is a property of single-token decoding rather than of any particular dataset, and that it holds on public data where results can be independently reproduced.

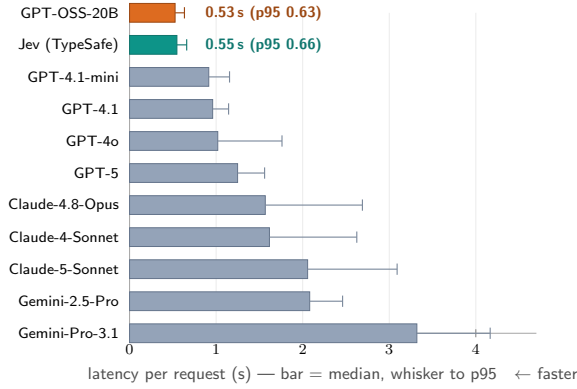


Figure 5: End-to-end per-request serving latency on SST-2 (872 examples), median (p50) with a whisker to the p95 tail, measured under the same round-robin, concurrency-1, shared serving-interface protocol as Figure 2. Orange = our Koa-action single-token classifier; teal = the external single-token system Jev/TypeSafe; slate = frontier LLMs. The two single-token systems (Koa-action 0.53 s, Jev 0.55 s) are tied at the front and well ahead of every frontier model, isolating the speedup to the single-token formulation itself.

B Additional Benchmarks

To complement the classical text benchmarks used in the main paper (SST-2, Amazon Reviews), we further evaluate Koa-action on two modern and widely-used datasets (Muennighoff et al., 2022; Enevoldsen et al., 2025): **TweetTopic** (6 classes) (Antypas et al., 2022) and **Banking77** (77 classes) (Casanueva et al., 2020). These datasets are substantially more challenging. To show that the approach is not tied to a single backbone, we report Koa-action applied to three different base models—Mistral-3-24B, GPT-OSS-20B, and Gemma-4-26B. As shown in Table 3, every Koa-action variant is competitive with GPT-4o while preserving the single-token decoding advantage that keeps its latency well below decoder-based generation or GPT-4o; on the 77-class Banking77 task the GPT-OSS-20B and Gemma-4-26B backbones both surpass GPT-4o. These results further highlight the practicality of Koa-action for latency-sensitive text classification.

Multilingual intent classification (MTOP). To strengthen our evaluation beyond English-only settings, we further evaluate Koa-action on MTOP (Li et al., 2021), a widely used multilingual intent-classification benchmark covering six languages (English, German, Spanish, French, Hindi, Thai). The results are in Table 4.

Since our fine-tuning corpus is entirely English, Koa-action naturally achieves lower multilingual accuracy than GPT-4o and the multilingual Mistral base model. The gap is backbone-dependent: the Gemma-4-26B and GPT-OSS-20B backbones (average 73.7 and 70.6) generalize noticeably better across languages than the Mistral-3-24B fine-tune (68.7), and they avoid its sharp drop on Thai (59.3 vs. 68–73), indicating that the low-resource degradation reflects the English-only supervision rather than the single-token formulation itself. Across all backbones Koa-action retains its key advantage—single-token decoding preserves the low-latency inference of our other benchmarks—highlighting that our classification formulation scales efficiently even in multilingual settings. We expect multilingual accuracy to improve substantially with multilingual supervision, which we identify as an important direction for future work.

C Ablation Study

To assess the value of incorporating both unimodal and multimodal supervision, we evaluated our fine-tuned Gemma-3-27B model on the held-out intent-routing dataset. When adapted only on multimodal data, the model achieved 80.6% accuracy. By contrast, when adaptation combined both text-only and multimodal data, accuracy improved to 84.7% at the same single-token decoding cost, i.e. without any latency penalty. These results suggest that exposing the model to both text and multimodal supervision during adaptation provides stronger representations and leads to consistent accuracy gains. A similar improvement ($\approx 0.5\%$) at unchanged latency was also observed on DBpedia, indicating that the effect generalizes beyond a single dataset.

D Robustness and Generalization Analysis

D.1 Effect of Label-Set Sizes

We further analyze the impact of the number of label sets on model performance by evaluating across datasets with increasing topic sizes, ranging from binary sentiment classification (SST-2, Amazon Reviews) to multi-class categorization (DBpedia with 14 topics).

Generalization ability. As shown in Figure 6a, our fine-tuned Gemma-3-27B model consistently maintains high accuracy across datasets of varying difficulty. Even as the label space expands from 2

Dataset	Model	Acc. (%)
TweetTopic	GPT-4o	77.50
	Mistral-3-24B (Base)	78.13
	Mistral-3-24B (FT, Koa-action)	81.86
	GPT-OSS-20B (FT, Koa-action)	77.42
	Gemma-4-26B (FT, Koa-action)	79.43
Banking77	GPT-4o	74.68
	Mistral-3-24B (Base)	74.74
	Mistral-3-24B (FT, Koa-action)	70.94
	GPT-OSS-20B (FT, Koa-action)	79.21
	Gemma-4-26B (FT, Koa-action)	77.28

Table 3: Evaluation results on TweetTopic and Banking77 across three Koa-action backbones (Mistral-3-24B, GPT-OSS-20B, Gemma-4-26B). All Koa-action variants are competitive with GPT-4o while retaining single-token decoding; on Banking77 the GPT-OSS-20B and Gemma-4-26B backbones both surpass GPT-4o.

Model	en	de	es	fr	hi	th	Avg
GPT-4o	79.03	78.26	77.02	79.22	74.39	78.28	77.70
Mistral-3-24B (Base)	81.08	79.29	81.65	80.18	77.45	70.81	78.41
Mistral-3-24B (FT, Koa-action)	70.63	69.71	74.05	71.12	67.55	59.31	68.73
GPT-OSS-20B (FT, Koa-action)	71.99	70.62	71.84	71.73	68.90	68.46	70.59
Gemma-4-26B (FT, Koa-action)	75.06	70.75	73.95	76.45	73.07	72.73	73.67

Table 4: Multilingual evaluation on the MTOP intent classification benchmark (6 languages) across three Koa-action backbones.

to 14 categories, the accuracy remains 95% on DBpedia and comparable to binary sentiment datasets, demonstrating strong zero-shot generalization.

Efficiency stability. In addition to accuracy, we examine efficiency via P50 latency. The results reveal that latency remains essentially flat as the label space grows, since the model still emits a single control token regardless of how many labels are in play. This indicates that our design achieves scalable inference efficiency while handling tasks of increasing complexity.

D.2 Effect of Batch Size

This section examines whether the latency benefits of Koa-action persist under larger batch sizes, a common setting in practical deployment scenarios (e.g., vLLM-based serving). While the main experiments fix the batch size to 1 for controlled comparison, real-world systems frequently operate with significantly larger batches to maximize throughput. To assess robustness under such conditions, we measure per-sample median (P50) and tail (P95) latencies for batch sizes ranging from 2 to 64 on Amazon Reviews using Gemma-3-27B.

Across all batch sizes, Koa-action consistently achieves substantially lower latency than the base model. Notably, the improvement remains pronounced even in high-throughput regimes (e.g., batch size 64), where inference engines are typically most optimized. These results demonstrate that the single-token decision mechanism of Koa-action yields durable efficiency gains that are insensitive to batching.

Amazon Reviews. Table 5 reports per-sample latency across batch sizes. Koa-action maintains a clear advantage over the base model.

We observe that per-sample latency generally decreases as batch size increases. Additionally, these results confirm that the latency improvements of Koa-action are not limited to single-example inference, but extend reliably to the large-batch regimes commonly used for high-throughput deployment.

D.3 Zero-Shot Adaptation via Control-Token Permutations

This section examines the model’s ability to generalize to unseen mappings between semantic labels and control tokens. During training, the correspon-

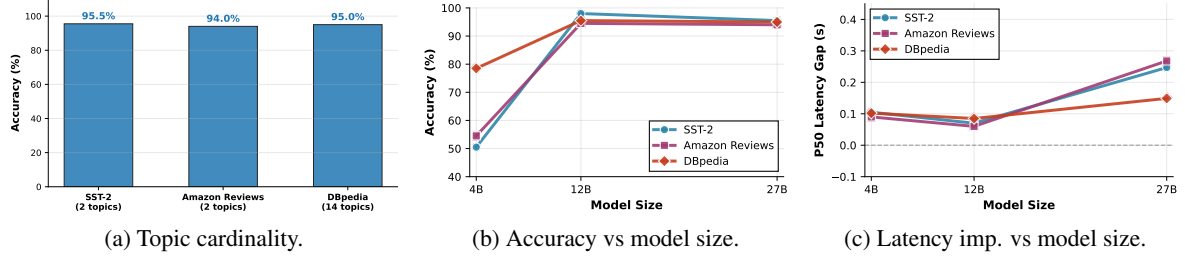


Figure 6: Performance of Gemma-3 models across datasets.

Batch	Base P50	Koa-action P50 (improv.)	Base P95	Koa-action P95 (improv.)
2	0.1106	0.0545 (0.0561 ↓)	0.1175	0.0875 (0.0300 ↓)
4	0.0554	0.0327 (0.0227 ↓)	0.0580	0.0427 (0.0153 ↓)
8	0.0282	0.0205 (0.0077 ↓)	0.0295	0.0222 (0.0073 ↓)
16	0.0146	0.0108 (0.0038 ↓)	0.0148	0.0110 (0.0038 ↓)
32	0.0090	0.0068 (0.0022 ↓)	0.0098	0.0069 (0.0029 ↓)
64	0.0269	0.0056 (0.0213 ↓)	0.0365	0.0053 (0.0312 ↓)

Table 5: Amazon Reviews — Per-sample latency (seconds) vs. batch size for Gemma-3-27B.

dence between labels and control tokens is randomized across instances, preventing the model from relying on fixed token identities. To assess whether the model can operate under entirely new mappings at inference time, we evaluate the fine-tuned Koa-action model on MIntRec 2.0 while applying random permutations of the control-token assignments.

In each of ten runs, a set of 30 control tokens is sampled from the 500-token pool and assigned to the 30 intent labels according to a fresh random permutation. The fine-tuned Gemma-3-27B model is then evaluated on the full test set (2,033 samples, with video) using this new mapping. This procedure ensures that every run tests the model under a previously unseen association between output tokens and semantic categories. Accuracy remains stable across permutations and on par with the fixed-mapping result: the mean accuracy is **61.62%** with a standard deviation of **1.11**, close to the **62.72%** reported for the same model in Table 1. This indicates that the model interprets the label descriptions provided in the prompt rather than memorizing token identities. Table 6 summarizes the results.

Procedure for swapping label sets at inference.

Adapting Koa-action to a new label set at inference time requires no retraining, adapter swapping, or model modification. The operator updates the system prompt to list the new label names and their natural-language descriptions, each mapped to a reserved control token ([control_1], [control_2],

Run	Seed	Accuracy (%)
1	42	61.93
2	43	59.17
3	44	60.90
4	45	61.49
5	46	63.40
6	47	61.24
7	48	61.98
8	49	60.99
9	50	62.76
10	51	62.32
Mean $\pm$ Std		61.62 $\pm$ 1.11

Table 6: Accuracy under random permutations of control-token assignments during inference (Gemma-3-27B, full MIntRec 2.0 test set). Accuracy stays on par with the fixed-mapping result (62.72%, Table 1).

...), following the same template used during training (Appendix G). The decision space is then restricted to the control tokens for the active labels, and the model emits one of them in a single decode step, conditioning on the descriptions in the prompt rather than on fixed token identities. Because the mapping between labels and tokens is randomized during training, any assignment of labels to reserved tokens is valid at inference; the permutation results above confirm that accuracy is stable under such reassignments. This makes label additions, merges, and deprecations—common in production intent inventories—a prompt-side edit rather than a retraining event.

D.4 Scaling Analysis

We further investigate how accuracy and efficiency scale with model sizes. Figures 6b and 6c report results for Gemma-3 models with 4B, 12B, and 27B parameters on three text benchmarks. **Accuracy.** Performance improves consistently as model size increases (Figure 6b). The 4B model shows lower accuracy, while the 12B model closes much of the gap. The 27B model achieves the strongest results across all datasets, exceeding 95% on SST-2, Amazon Reviews, and DBpedia. **Latency.** Figure 6c reports the *P50 latency improvement* of Koa-action relative to the corresponding base model. The improvement is positive across all datasets, showing that Koa-action consistently reduces latency compared to standard autoregressive decoding. The improvement is modest for the 4B and 12B models, but becomes clearly larger for the 27B model. This trend indicates that Koa-action’s efficiency advantage becomes more pronounced for larger models.

E Data Construction

E.1 Dataset Details

We construct a balanced adaptation corpus with comparable amounts of text-only and multimodal classification instances, each pairing an input with a categorical label and spanning diverse intent- and topic-classification domains. All instances are reformulated into a unified JSON schema (see Figure 7), with consistent messages fields and an optional image_path. To mitigate label memorization, we randomize control-token assignments, mapping class labels to tokens drawn from a reserved pool of up to 500 control tokens. The evaluation benchmarks reported in this paper are held out from this corpus.

E.2 Implement Details

We adapt each base model with supervised fine-tuning, keeping the token embeddings and LM head trainable so that the newly added control tokens can be learned. Training uses mixed precision (bfloat16) with gradient checkpointing on standard multi-GPU infrastructure, and early stopping on validation loss.

For LM-BFF, we use RoBERTa-base as the backbone, follow the standard prompt-based fine-tuning setup, and use 1 in-context demonstration with 16 training samples per class (consistent with the (Gao et al., 2020)). Template and verbalizer settings

Example of a Classification Training Instance

```
{
  "messages": [
    { "role": "system", "content": "You are a classification expert. Topics: [control_x] Complain, [control_y] Praise, [control_z] Apologise ..." },
    { "role": "user", "content": "### USER CONVERSATION HERE ###" },
    { "role": "assistant", "content": "[control_x]" }
  ],
  "image_path": "path/to/image.jpg",
}
```

Figure 7: Illustration of a fine-tuning training instance in our classification datasets. Each sample includes the structured messages field and optional image_path.

exactly match those reported in LM-BFF. As expected, LM-BFF performance is sensitive to the number of sampled demonstrations: using more samples improves accuracy but also increases latency substantially.

Baseline configurations and fairness. The baselines differ in the amount of task-specific adaptation, and we configure each following its intended usage. MAG-BERT and MulT are trained on the *full* MIntRec 2.0 training set (the benchmark’s standard protocol), while LM-BFF is run in its designed few-shot regime. For the BERT/RoBERTa linear heads we use 16-shot fine-tuning by design: these encoders require a task-specific head retrained from scratch whenever the label set changes, whereas Koa-action reuses a single adapter and adapts by editing the prompt—so training the heads on the full dataset would compare a per-task, retrained classifier against a general-purpose model on precisely the axis where the two paradigms differ. The commercial APIs (GPT-4o, GPT-5) do not expose fine-tuning and serve as strong out-of-the-box references. We make these differences explicit so that the single-token formulation is not conflated with the effect of supervised fine-tuning alone.

F Preliminary Study

F.1 Alternative Prompting Baselines

To examine whether simple prompting strategies can emulate the single-symbol classification behavior of our method, we evaluate alternative formulations of the form $[x, \text{cls}, \text{cls-description}]$ using

GPT-4o on the MIntRec2.0 benchmark. In this setting, the model is provided with the utterance, associated video frames, and the full mapping between the 30 intent classes and their corresponding indices. The prompt explicitly instructs the model to output only the integer associated with the predicted intent. An example prompt is shown in Figure 8.

Example of a Class-Index Prompting Instance (MIntRec2.0)

```
{
  "messages": [
    { "role": "system", "content": "You are an intent classification system. Respond with the number (0-29) corresponding to the intent. Mapping: 0: Acknowledge, 1: Advise, ..., 28: Thank, 29: Warn. Output the number (0-29):" },
    { "role": "user", "content": "Thank you" },
    { "role": "assistant", "content": "28" }
  ],
  "image_path": "data:image/jpeg;base64,..."
}
```

Figure 8: Illustration of the class-index prompting baseline evaluated on MIntRec2.0.

Despite the explicit numeric constraint, the model does not consistently produce a single-symbol output. In a non-trivial number of cases (16.67%), GPT-4o generates additional text or multi-token strings. For example, when the expected prediction is 4, the model sometimes outputs a full sentence such as “The intent of the utterance...”, or formats the answer as “3: Apologise” rather than emitting a single integer. Furthermore, integers above nine are represented as multi-token sequences under the GPT-4o tokenizer, which inherently increases decoding latency and introduces substantial variability in response length. These behaviors prevent the model from operating as a deterministic one-token classifier and stand in contrast to the stable single-token decoding enabled by our learned control-token interface. This analysis indicates that prompting-based alternatives are insufficient to achieve low-latency, single-token inference.

In terms of classification accuracy on MIntRec2.0, the class-index prompt described above (without constrained decoding) yields 45.12% accuracy, and adding constrained decoding over the 30 label tokens further increases it

to 45.56%. Thus, alternative prompting and constrained decoding slightly reduce formatting and parsing errors, but the gains are modest and do not close the substantial gap to Koa-action (e.g., 62.72% accuracy for Gemma-3-27B Koa-action in Table 1).

G Prompt Configuration

We provide representative prompt templates used for evaluation across datasets, as shown in Figures 9 to 12.

```
Example Prompt (SST-2)

{
  "messages": [
    { "role": "system", "content": "You are a classification expert. Topics: [control_1] Negative, [control_2] Positive. Based on the overall sentiment expressed in this review, respond with the relevant control token:" },
    { "role": "user", "content": "it's a charming and often affecting journey." },
    { "role": "assistant", "content": "[control_2]" }
  ],
  "image_path": "none",
}
```

Figure 9: Prompt template for SST-2 movie reviews.

```
Example Prompt (Amazon Reviews)

{
  "messages": [
    { "role": "system", "content": "You are a classification expert. Topics: [control_1] Negative, [control_2] Positive. Based on the overall sentiment expressed in this review, respond with the relevant control token:" },
    { "role": "user", "content": "DVD Player crapped out after one year. I also began having the incorrect disc problems that I've read about on here. The VCR still works, but the DVD side is useless. I understand that DVD players sometimes just quit on you, but after not even one year? To me that's a sign of bad quality. I'm giving up JVC after this as well. I'm sticking to Sony or giving another brand a shot." },
    { "role": "assistant", "content": "[control_1]" }
  ],
  "image_path": "none",
}
```

Figure 10: Illustration of an inference prompt from Amazon Reviews (Negative sentiment).

Example Prompt (DBpedia)

```
{
  "messages": [
    { "role": "system", "content": "You are a classification expert. Topics: [control_1] Company, [control_2] EducationalInstitution, [control_3] Artist, [control_4] Athlete, [control_5] OfficeHolder, [control_6] MeanOfTransportation, [control_7] Building, [control_8] NaturalPlace, [control_9] Village, [control_10] Animal, [control_11] Plant, [control_12] Album, [control_13] Film, [control_14] WrittenWork. Based on the content of this article, respond with the relevant control token:" },
    { "role": "user", "content": "Pizza Port Brewing Company is a brewpub with five locations in Southern California: Solana Beach, two in Carlsbad (Downtown and Bressi Ranch), Ocean Beach and San Clemente. A former Pizza Port location in San Marcos spun out of Pizza Port in 2006 and is now an independent operation, the Port Brewing Company / Lost Abbey brewery. It has received multiple awards, including Small Brewpub of the Year for both 2003 and 2004 by the Great American Beer Festival and six awards for its beers at the World Beer Cup." },
    { "role": "assistant", "content": "[control_1]" }
  ],
  "image_path": "none",
}
```

Figure 11: Illustration of an inference prompt from DBpedia (Company category).

Example Prompt (Topic Classification)

```
{
  "messages": [
    { "role": "system", "content": "You are a topic classification expert. Before making a decision, carefully follow all the topic-specific instructions/descriptions. Topics: [control_1] Acknowledge, [control_2] Advise, [control_3] Agree, [control_4] Apologise, [control_5] Arrange, [control_6] Ask for help, [control_7] Asking for opinions, [control_8] Care, [control_9] Comfort, [control_10] Complain, [control_11] Confirm, [control_12] Criticize, [control_13] Doubt, [control_14] Emphasize, [control_15] Explain, [control_16] Flaunt, [control_17] Greet, [control_18] Inform, [control_19] Introduce, [control_20] Invite, [control_21] Joke, [control_22] Leave, [control_23] Oppose, [control_24] Plan, [control_25] Praise, [control_26] Prevent, [control_27] Refuse, [control_28] Taunt, [control_29] Thank, [control_30] Warn. Based on the above conversation, respond with the relevant topic ID:" },
    { "role": "user", "content": "Thank you so much for your help! I really appreciate it." },
    { "role": "assistant", "content": "[control_29]" }
  ],
  "image_path": "none",
}
```

Figure 12: Illustration of a prompt for topic classification (Thank intent).